

Lab 4

Date : 24/08/2026

Assignment

Topic: SQL Clauses

Objective: To develop practical understanding of SQL clauses such as WHERE, ORDER BY, GROUP BY, HAVING, DISTINCT, LIKE, IN, BETWEEN, and LIMIT.

Case Study

Consider an **EMPLOYEE** table with the following structure:

Field	Description
Emp_ID	Employee ID
Emp_Name	Employee Name
Department	Department Name
Designation	Employee Designation
Salary	Monthly Salary
City	Employee City
Joining_Date	Date of Joining

Assignment Questions

1. Create the **EMPLOYEE** table with appropriate data types and constraints. Insert at least **15 employee records** belonging to different departments and cities.
2. Using the **WHERE clause**, display employees whose salary is greater than ₹50,000.
3. Use the **BETWEEN clause** to retrieve employees whose salary is between ₹40,000 and ₹70,000.
4. Using the **IN clause**, display employees working in the *IT*, *HR*, or *Finance* departments.
5. Apply the **LIKE clause** to find employees whose names start with the letter 'A'.
6. Use the **DISTINCT clause** to display all unique departments available in the **EMPLOYEE** table.
7. Using the **ORDER BY clause**, display employee details in descending order of salary.
8. Apply the **GROUP BY clause** to find the **number of employees in each department**.

9. Use GROUP BY with an aggregate function to calculate the **average salary of employees in each department**.
10. Apply the **HAVING clause** to display only those departments whose average salary is greater than ₹50,000.
11. Write a query using **WHERE, GROUP BY, HAVING, and ORDER BY together** to identify departments that:
 - Have employees earning more than ₹40,000,
 - Have at least 2 such employees, and
 - Display the results according to the number of employees in descending order.
1. Prepare a brief comparison explaining the difference between **WHERE and HAVING clauses**, supported by SQL queries executed on the EMPLOYEE table.

Assignment Questions

1. Create the **EMPLOYEE** table with appropriate data types and constraints. Insert at least **15 employee records** belonging to different departments and cities.

Code

-- 1. Create EMPLOYEE table and insert 15 records

```
CREATE DATABASE employee_db;
```

```
USE employee_db;
```

-- Create table

```
CREATE TABLE EMPLOYEE (  
    Emp_ID INT PRIMARY KEY,  
    Emp_Name VARCHAR(50) NOT NULL,  
    Department VARCHAR(30) NOT NULL,  
    Designation VARCHAR(50),  
    Salary DECIMAL(10,2) CHECK (Salary > 0),  
    City VARCHAR(30),  
    Joining_Date DATE  
);
```

-- Insert 15 employee records

```
INSERT INTO EMPLOYEE
```

```
(Emp_ID, Emp_Name, Department, Designation, Salary, City, Joining_Date)  
VALUES
```

```
(101, 'Amit Sharma', 'IT', 'Software Engineer', 65000, 'Mumbai', '2022-01-15'),
```

```
(102, 'Priya Singh', 'HR', 'HR Executive', 45000, 'Delhi', '2021-06-20'),
```

```
(103, 'Rahul Verma', 'Finance', 'Financial Analyst', 58000, 'Bangalore', '2020-03-10'),
```

```
(104, 'Ananya Gupta', 'IT', 'Data Analyst', 72000, 'Pune', '2023-02-18'),
```

```
(105, 'Vikas Kumar', 'Sales', 'Sales Executive', 38000, 'Ranchi', '2022-08-12'),
```

```
(106, 'Sneha Patel', 'Finance', 'Accountant', 48000, 'Ahmedabad', '2021-11-05'),
```

```
(107, 'Arjun Mehta', 'IT', 'Backend Developer', 68000, 'Mumbai', '2022-04-25'),
```

```
(108, 'Neha Roy', 'HR', 'Recruiter', 52000, 'Kolkata', '2020-09-15'),
```

(109, 'Rohan Das', 'Marketing', 'Marketing Executive', 42000, 'Chennai', '2023-01-10'),

(110, 'Ayesha Khan', 'Finance', 'Finance Manager', 75000, 'Hyderabad', '2019-07-22'),

(111, 'Karan Joshi', 'IT', 'Frontend Developer', 55000, 'Delhi', '2021-02-14'),

(112, 'Pooja Shah', 'Sales', 'Sales Manager', 62000, 'Mumbai', '2020-05-30'),

(113, 'Aditya Rao', 'Marketing', 'Marketing Manager', 67000, 'Pune', '2019-12-01'),

(114, 'Meera Nair', 'HR', 'HR Manager', 70000, 'Bangalore', '2018-10-19'),

(115, 'Akash Jain', 'Finance', 'Senior Accountant', 56000, 'Jaipur', '2022-07-08');

-- check the records

SELECT * FROM EMPLOYEE;

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that inserts 15 rows into the EMPLOYEE table and then checks the records. The query is as follows:

```

(106, 'Sneha Patel', 'Finance', 'Accountant', 48000, 'Ahmedabad', '2021-11-05'),
(107, 'Arjun Mehta', 'IT', 'Backend Developer', 68000, 'Mumbai', '2022-04-25'),
(108, 'Neha Roy', 'HR', 'Recruiter', 52000, 'Kolkata', '2020-09-15'),
(109, 'Rohan Das', 'Marketing', 'Marketing Executive', 42000, 'Chennai', '2023-01-10'),
(110, 'Ayesha Khan', 'Finance', 'Finance Manager', 75000, 'Hyderabad', '2019-07-22'),
(111, 'Karan Joshi', 'IT', 'Frontend Developer', 55000, 'Delhi', '2021-02-14'),
(112, 'Pooja Shah', 'Sales', 'Sales Manager', 62000, 'Mumbai', '2020-05-30'),
(113, 'Aditya Rao', 'Marketing', 'Marketing Manager', 67000, 'Pune', '2019-12-01'),
(114, 'Meera Nair', 'HR', 'HR Manager', 70000, 'Bangalore', '2018-10-19'),
(115, 'Akash Jain', 'Finance', 'Senior Accountant', 56000, 'Jaipur', '2022-07-08');

-- check the records
SELECT * FROM EMPLOYEE;

```

The Result Grid shows the following data:

Emp_ID	Emp_Name	Department	Designation	Salary	City	Joining_Date
101	Amit Sharma	IT	Software Engineer	65000.00	Mumbai	2022-01-15
102	Priya Singh	HR	HR Executive	45000.00	Delhi	2021-06-20
103	Rahul Verma	Finance	Financial Analyst	58000.00	Bangalore	2020-03-10
104	Ananya Gupta	IT	Data Analyst	72000.00	Pune	2023-02-18
105	Vikas Kumar	Sales	Sales Executive	38000.00	Ranchi	2022-08-12
106	Sneha Patel	Finance	Accountant	48000.00	Ahmedabad	2021-11-05
107	Arjun Mehta	IT	Backend Developer	68000.00	Mumbai	2022-04-25
108	Neha Roy	HR	Recruiter	52000.00	Kolkata	2020-09-15
109	Rohan Das	Marketing	Marketing Executive	42000.00	Chennai	2023-01-10
110	Ayesha Khan	Finance	Finance Manager	75000.00	Hyderabad	2019-07-22
111	Karan Joshi	IT	Frontend Developer	55000.00	Delhi	2021-02-14
112	Pooja Shah	Sales	Sales Manager	62000.00	Mumbai	2020-05-30
113	Aditya Rao	Marketing	Marketing Manager	67000.00	Pune	2019-12-01
114	Meera Nair	HR	HR Manager	70000.00	Bangalore	2018-10-19
115	Akash Jain	Finance	Senior Accountant	56000.00	Jaipur	2022-07-08

The Action Output shows the following actions:

Time	Action	Response	Duration / Fetch Time
15:50:53	CREATE DATABASE employee_db	1 row(s) affected	0.015 sec
15:55:57	CREATE TABLE EMPLOYEE (Emp_ID INT PRIMARY KEY, Emp_Name VARCHAR(50) NOT NULL, Department VARCHAR(30) NOT NULL, Designatio...	0 row(s) affected	0.032 sec
15:56:19	CREATE DATABASE employee_db	Error Code: 1007. Can't create database 'employee_d...	0.0033 sec
15:56:21	INSERT INTO EMPLOYEE (Emp_ID, Emp_Name, Department, Designation, Salary, City, Joining_Date) VALUES (101, 'Amit Sharma', 'IT', 'Software Engineer',...	15 row(s) affected Records: 15 Duplicates: 0 Warnin...	0.017 sec
15:57:05	SELECT * FROM EMPLOYEE LIMIT 0, 1000	15 row(s) returned	0.0007 sec / 0.00054...

2. Using the **WHERE clause**, display employees whose salary is greater than ₹50,000.

Code

-- Display employees whose salary is greater than ₹50,000.

SELECT *

FROM EMPLOYEE

WHERE Salary > 50000;

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'employee_db' selected. The main editor shows a SQL query with line numbers 35 to 57. The query is as follows:

```
35  
36 (109, 'Rohan Das', 'Marketing', 'Marketing Executive', 42000, 'Chennai', '2023-01-10'),  
37  
38 (110, 'Ayesha Khan', 'Finance', 'Finance Manager', 75000, 'Hyderabad', '2019-07-22'),  
39  
40 (111, 'Karan Joshi', 'IT', 'Frontend Developer', 55000, 'Delhi', '2021-02-14'),  
41  
42 (112, 'Pooja Shah', 'Sales', 'Sales Manager', 62000, 'Mumbai', '2020-05-30'),  
43  
44 (113, 'Aditya Rao', 'Marketing', 'Marketing Manager', 67000, 'Pune', '2019-12-01'),  
45  
46 (114, 'Meera Nair', 'HR', 'HR Manager', 70000, 'Bangalore', '2018-10-19'),  
47  
48 (115, 'Akash Jain', 'Finance', 'Senior Accountant', 56000, 'Jaipur', '2022-07-08');  
49  
50  
51 -- check the records  
52 SELECT * FROM EMPLOYEE;  
53  
54 -- Display employees whose salary is greater than ₹50,000.  
55 SELECT *  
56 FROM EMPLOYEE  
57 WHERE Salary > 50000;
```

Below the query editor, the 'Result Grid' shows the results of the query. It contains 11 rows of employee data. The columns are: Emp_ID, Emp_Name, Department, Designation, Salary, City, and Joining_Date.

Emp_ID	Emp_Name	Department	Designation	Salary	City	Joining_Date
101	Amit Sharma	IT	Software Engineer	65000.00	Mumbai	2022-01-15
103	Abhinav Verma	Finance	Financial Analyst	58000.00	Mangalore	2020-03-10
104	Ananya Gupta	IT	Data Analyst	72000.00	Pune	2023-02-18
107	Arjun Mehta	IT	Backend Developer	68000.00	Mumbai	2022-04-25
108	Alina Roy	HR	Recruiter	52000.00	Kolkata	2020-09-15
110	Ayesha Khan	Finance	Finance Manager	75000.00	Hyderabad	2019-07-22
111	Karan Joshi	IT	Frontend Developer	55000.00	Delhi	2021-02-14
112	Pooja Shah	Sales	Sales Manager	62000.00	Mumbai	2020-05-30
113	Aditya Rao	Marketing	Marketing Manager	67000.00	Pune	2019-12-01
114	Meera Nair	HR	HR Manager	70000.00	Bangalore	2018-10-19
115	Akash Jain	Finance	Senior Accountant	56000.00	Jaipur	2022-07-08

At the bottom, the 'Action Output' tab shows the execution log of the queries, including the successful execution of the SELECT query with 11 rows returned.

3. Use the **BETWEEN clause** to retrieve employees whose salary is between ₹40,000 and ₹70,000.

Code

-- Retrieve employees whose salary is between ₹40,000 and ₹70,000.

SELECT *

FROM EMPLOYEE

WHERE Salary BETWEEN 40000 AND 70000;

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'employee_db' selected. The main editor contains a SQL query with comments and a WHERE clause using the BETWEEN operator. Below the query, the 'Result Grid' shows a table of employee data. The 'Action Output' pane at the bottom displays the execution log for the queries.

```
40 (111, 'Karan Joshi', 'IT', 'Frontend Developer', 55000, 'Delhi', '2021-02-14'),
41
42 (112, 'Pooja Shah', 'Sales', 'Sales Manager', 62000, 'Mumbai', '2020-05-30'),
43
44 (113, 'Aditya Rao', 'Marketing', 'Marketing Manager', 67000, 'Pune', '2019-12-01'),
45
46 (114, 'Meera Nair', 'HR', 'HR Manager', 70000, 'Bangalore', '2018-10-19'),
47
48 (115, 'Akash Jain', 'Finance', 'Senior Accountant', 56000, 'Jaipur', '2022-07-08');
49
50
51 -- check the records
52 * SELECT * FROM EMPLOYEE;
53
54 -- Display employees whose salary is greater than ₹50,000.
55 * SELECT *
56 FROM EMPLOYEE
57 WHERE Salary > 50000;
58
59 -- Retrieve employees whose salary is between ₹40,000 and ₹70,000.
60 * SELECT *
61 FROM EMPLOYEE
62 WHERE Salary BETWEEN 40000 AND 70000;
```

Emp_ID	Emp_Name	Department	Designation	Salary	City	Joining_Date
101	Amit Sharma	IT	Software Engineer	55000.00	Mumbai	2022-01-15
102	Priya Singh	HR	HR Executive	45000.00	Delhi	2021-06-20
103	Rahul Verma	Finance	Financial Analyst	58000.00	Bangalore	2020-03-10
106	Smriti Patel	Finance	Accountant	48000.00	Amritabad	2021-11-25
107	Ajay Mehta	IT	Backend Developer	68000.00	Mumbai	2023-04-25
108	Neha Roy	HR	Recruiter	52000.00	Kolkata	2020-09-15
109	Rachan Datta	Marketing	Marketing Executive	73000.00	Phnom	2022-01-15

Time	Action	Response	Duration / Fetch Time
15:50:53	CREATE DATABASE employee_db	1 row(s) affected	0.015 sec
15:55:57	CREATE TABLE EMPLOYEE (Emp_ID INT PRIMARY KEY, Emp_Name VARCHAR(50) NOT NULL, Department VARCHAR(30) NOT NULL, Designatio...	0 row(s) affected	0.032 sec
15:56:19	CREATE DATABASE employee_db	Error Code: 1007, Can't create database 'employee_d...	0.0033 sec
15:56:21	INSERT INTO EMPLOYEE (Emp_ID, Emp_Name, Department, Designation, Salary, City, Joining_Date) VALUES (101, 'Amit Sharma', 'IT', 'Software Engineer',...	15 row(s) affected Records: 15 Duplicates: 0 Warnin...	0.017 sec
15:57:05	SELECT * FROM EMPLOYEE LIMIT 0, 1000	15 row(s) returned	0.0087 sec / 0.00054...
16:01:48	SELECT * FROM EMPLOYEE WHERE Salary > 50000 LIMIT 0, 1000	11 row(s) returned	0.0054 sec / 0.00012...
16:03:29	SELECT * FROM EMPLOYEE WHERE Salary BETWEEN 40000 AND 70000 LIMIT 0, 1000	12 row(s) returned	0.010 sec / 0.00046...

Query Completed

4. Using the **IN clause**, display employees working in the *IT*, *HR*, or *Finance* departments.

Code

-- Display employees working in IT, HR, or Finance.

SELECT *

FROM EMPLOYEE

WHERE Department IN ('IT', 'HR', 'Finance');

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
-- check the records
SELECT * FROM EMPLOYEE;

-- Display employees whose salary is greater than ₹50,000.
SELECT *
FROM EMPLOYEE
WHERE Salary > 50000;

-- Retrieve employees whose salary is between ₹40,000 and ₹70,000.
SELECT *
FROM EMPLOYEE
WHERE Salary BETWEEN 40000 AND 70000;

-- Display employees working in IT, HR, or Finance.
SELECT *
FROM EMPLOYEE
WHERE Department IN ('IT', 'HR', 'Finance');
```

The Results Grid shows the following data:

Emp_ID	Emp_Name	Department	Designation	Salary	City	Joining_Date
101	Amit Sharma	IT	Software Engineer	65000.00	Mumbai	2022-01-15
102	Priya Singh	HR	HR Executive	45000.00	Delhi	2021-08-20
103	Rahul Verma	Finance	Financial Analyst	58000.00	Bangalore	2023-03-10
104	Ananya Gupta	IT	Data Analyst	72000.00	Pune	2023-02-18
106	Sneha Patel	Finance	Accountant	48000.00	Ahmedabad	2021-11-05
107	Aryan Mehta	IT	Backend Developer	68000.00	Mumbai	2022-04-25
108	Rishabh Singh	HR	HR Manager	78000.00	Kolkata	2020-06-14

The Action Output shows the following log:

Time	Action	Response	Duration / Fetch Time
15:55:57	CREATE TABLE EMPLOYEE (Emp_ID INT PRIMARY KEY, Emp_Name VARCHAR(50) NOT NULL, Department VARCHAR(50) NOT NULL, Designation VARCHAR(50) NOT NULL, Salary DECIMAL(10,2) NOT NULL, City VARCHAR(50) NOT NULL, Joining_Date DATE NOT NULL);	0 row(s) affected	0.032 sec
15:56:19	CREATE DATABASE employee_db	Error Code: 1007. Can't create database 'employee_d_'	0.0033 sec
15:56:21	INSERT INTO EMPLOYEE (Emp_ID, Emp_Name, Department, Designation, Salary, City, Joining_Date) VALUES (101, 'Amit Sharma', 'IT', 'Software Engineer', 65000.00, 'Mumbai', '2022-01-15');	15 row(s) affected Records: 15 Duplicates: 0 Warnings: 0	0.017 sec
15:57:05	SELECT * FROM EMPLOYEE LIMIT 0, 1000	15 row(s) returned	0.0067 sec / 0.00054...
16:01:48	SELECT * FROM EMPLOYEE WHERE Salary > 50000 LIMIT 0, 1000	11 row(s) returned	0.0054 sec / 0.00012...
16:03:29	SELECT * FROM EMPLOYEE WHERE Salary BETWEEN 40000 AND 70000 LIMIT 0, 1000	12 row(s) returned	0.010 sec / 0.00046...
16:04:46	SELECT * FROM EMPLOYEE WHERE Department IN ('IT', 'HR', 'Finance') LIMIT 0, 1000	11 row(s) returned	0.0080 sec / 0.00017...

5. Apply the **LIKE** clause to find employees whose names start with the letter 'A'.

Code

-- Find employees whose names start with the letter A.

SELECT *

FROM EMPLOYEE

WHERE Emp_Name LIKE 'A%';

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'employee_db' selected. The main editor window contains a SQL query with several comments and clauses. The query is as follows:

```
57 WHERE Salary > 50000;
58
59 -- Retrieve employees whose salary is between 40,000 and 70,000.
60 SELECT *
61 FROM EMPLOYEE
62 WHERE Salary BETWEEN 40000 AND 70000;
63
64 -- Display employees working in IT, HR, or Finance.
65 SELECT *
66 FROM EMPLOYEE
67 WHERE Department IN ('IT', 'HR', 'Finance');
68
69 SELECT *
70 FROM EMPLOYEE
71 WHERE Department = 'IT'
72 OR Department = 'HR'
73 OR Department = 'Finance';
74
75 -- Find employees whose names start with the letter A.
76 SELECT *
77 FROM EMPLOYEE
78 WHERE Emp_Name LIKE 'A%';
```

Below the query editor, the 'Result Grid' shows the results of the last query (line 78). The grid has columns: Emp_ID, Emp_Name, Department, Designation, Salary, City, and Joining_Date. The results are as follows:

Emp_ID	Emp_Name	Department	Designation	Salary	City	Joining_Date
101	Amit Sharma	IT	Software Engineer	65000.00	Mumbai	2022-01-15
104	Ananya Gupta	IT	Data Analyst	72000.00	Pune	2023-02-18
107	Arijit Mehta	IT	Backend Developer	68000.00	Mumbai	2022-04-25
110	Ayesha Khan	Finance	Finance Manager	75000.00	Hyderabad	2019-07-22
113	Aditya Rao	Marketing	Marketing Manager	87000.00	Pune	2018-12-01
115	Aash Jain	Finance	Senior Accountant	55000.00	Rajpur	2022-07-08

At the bottom, the 'Action Output' tab shows the execution log of the queries. The log includes the time, action, response, and duration for each query executed.

Time	Action	Response	Duration / Fetch Time
15:56:19	CREATE DATABASE employee_db	Error Code: 1007: Can't create database 'employee_d...	0.0033 sec
15:56:21	INSERT INTO EMPLOYEE (Emp_ID, Emp_Name, Department, Designation, Salary, City, Joining_Date) VALUES (101, 'Amit Sharma', 'IT', 'Software Engineer',...	15 row(s) affected Records: 15 Duplicates: 0 Warnin...	0.017 sec
15:57:05	SELECT * FROM EMPLOYEE LIMIT 0, 1000	15 row(s) returned	0.0067 sec / 0.00054...
16:01:48	SELECT * FROM EMPLOYEE WHERE Salary > 50000 LIMIT 0, 1000	11 row(s) returned	0.0054 sec / 0.00012...
16:03:29	SELECT * FROM EMPLOYEE WHERE Salary BETWEEN 40000 AND 70000 LIMIT 0, 1000	12 row(s) returned	0.010 sec / 0.00044...
16:04:46	SELECT * FROM EMPLOYEE WHERE Department IN ('IT', 'HR', 'Finance') LIMIT 0, 1000	11 row(s) returned	0.0060 sec / 0.00017...
16:06:47	SELECT * FROM EMPLOYEE WHERE Emp_Name LIKE 'A%' LIMIT 0, 1000	6 row(s) returned	0.0055 sec / 0.00019...

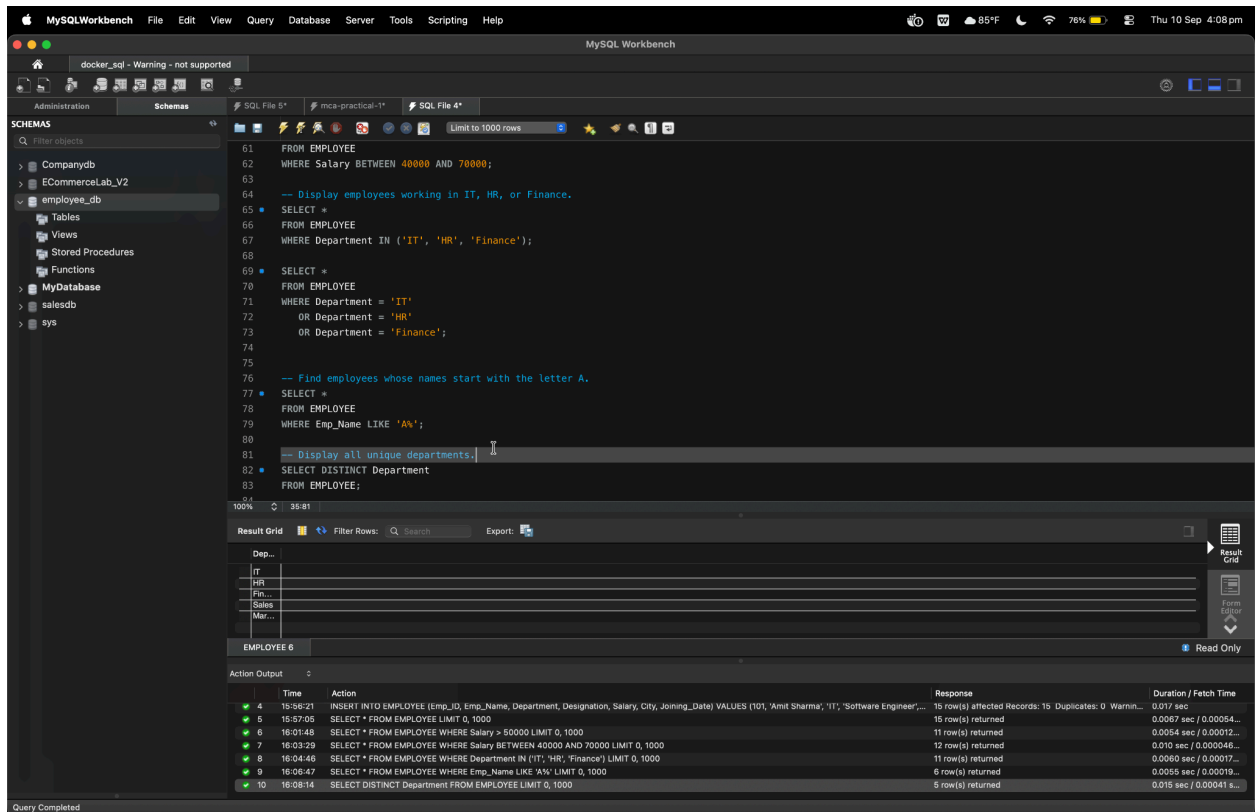
6. Use the **DISTINCT** clause to display all unique departments available in the EMPLOYEE table.

Code

-- Display all unique departments.

SELECT DISTINCT Department

FROM EMPLOYEE;



7. Using the **ORDER BY clause**, display employee details in descending order of salary.

Code

-- Display employee details in descending order of salary.

SELECT *

FROM EMPLOYEE

ORDER BY Salary DESC;

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'employee_db' selected. The main editor window contains the following SQL query:

```
68
69 SELECT *
70 FROM EMPLOYEE
71 WHERE Department = 'IT'
72 OR Department = 'HR'
73 OR Department = 'Finance';
74
75 -- Find employees whose names start with the letter A.
76
77 SELECT *
78 FROM EMPLOYEE
79 WHERE Emp_Name LIKE 'A%';
80
81 -- Display all unique departments.
82 SELECT DISTINCT Department
83 FROM EMPLOYEE;
84
85 -- Display employee details in descending order of salary.
86 SELECT *
87 FROM EMPLOYEE
88 ORDER BY Salary DESC;
89
90
```

The 'Result Grid' at the bottom shows the results of the last query (SELECT * FROM EMPLOYEE ORDER BY Salary DESC). The results are as follows:

Emp_ID	Emp_Name	Dep...	Designation	Salary	City	Joining_Date
110	Ayesha Khan	Fin...	Finance Manager	75000.00	Hyderabad	2019-07-22
104	Ananya Gupta	IT	Data Analyst	70000.00	Pune	2023-02-18
114	Ahmed Nair	HR	HR Manager	70000.00	Bangalore	2018-10-19
107	Arjun Mehta	IT	Backend Developer	68000.00	Mumbai	2022-04-25
113	Aditya Rao	Mar...	Marketing Manager	67000.00	Pune	2019-12-01
101	Amit Sharma	IT	Software Engineer	60000.00	Mumbai	2022-01-15
111	Divya Shah	Sales	Sales Manager	55000.00	Mumbai	2020-06-01

The 'Action Output' pane at the bottom shows the execution log of the queries, including the final query: 'SELECT * FROM EMPLOYEE ORDER BY Salary DESC LIMIT 0, 1000' which returned 15 rows in 0.0059 seconds.

8. Apply the **GROUP BY** clause to find the **number of employees in each department**.

Code

-- Find the number of employees in each department. Use COUNT() with GROUP BY.

SELECT Department, COUNT(*) AS Employee_Count

FROM EMPLOYEE

GROUP BY Department;

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'employee_db' selected. The main editor window contains a SQL query with several comments and a final query to find the number of employees in each department. The 'Result Grid' at the bottom shows the output of the final query, displaying a table with two columns: 'Dep...' and 'Employee_Count'. The 'Action Output' pane at the bottom shows the execution log for the queries.

```
72 OR Department = 'HR';
73 OR Department = 'Finance';
74
75
76 -- Find employees whose names start with the letter A.
77 SELECT +
78 FROM EMPLOYEE
79 WHERE Emp_Name LIKE 'A%';
80
81 -- Display all unique departments.
82 SELECT DISTINCT Department
83 FROM EMPLOYEE;
84
85 -- Display employee details in descending order of salary.
86 SELECT +
87 FROM EMPLOYEE
88 ORDER BY Salary DESC;
89
90
91 -- Find the number of employees in each department. Use COUNT() with GROUP BY.
92 SELECT Department, COUNT(*) AS Employee_Count
93 FROM EMPLOYEE
94 GROUP BY Department;
```

Dep...	Employee_Count
IT	4
HR	3
Fin...	4
Sales	2
Mar...	2

Time	Action	Response	Duration / Fetch Time
16:01:48	SELECT * FROM EMPLOYEE WHERE Salary > 50000 LIMIT 0, 1000	11 row(s) returned	0.0054 sec / 0.00012...
16:03:29	SELECT * FROM EMPLOYEE WHERE Salary BETWEEN 40000 AND 70000 LIMIT 0, 1000	12 row(s) returned	0.010 sec / 0.00046...
16:04:46	SELECT * FROM EMPLOYEE WHERE Department IN ('IT', 'HR', 'Finance') LIMIT 0, 1000	11 row(s) returned	0.0060 sec / 0.00017...
16:06:47	SELECT * FROM EMPLOYEE WHERE Emp_Name LIKE 'A%' LIMIT 0, 1000	6 row(s) returned	0.0056 sec / 0.00019...
16:08:14	SELECT DISTINCT Department FROM EMPLOYEE LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.00041...
16:09:52	SELECT * FROM EMPLOYEE ORDER BY Salary DESC LIMIT 0, 1000	15 row(s) returned	0.0059 sec / 0.00015...
16:10:47	SELECT Department, COUNT(*) AS Employee_Count FROM EMPLOYEE GROUP BY Department LIMIT 0, 1000	5 row(s) returned	0.022 sec / 0.00069...

9. Use GROUP BY with an aggregate function to calculate the **average salary of employees in each department**.

Code

-- Calculate the average salary of employees in each department.

SELECT Department,

AVG(Salary) AS Average_Salary

FROM EMPLOYEE

GROUP BY Department;

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'employee_db' selected. The main editor contains a SQL script with several queries. The query at line 97 is highlighted, which calculates the average salary for each department. Below the editor, the 'Result Grid' shows the output of this query. The 'Action Output' pane at the bottom displays a log of executed queries and their execution times.

Dep...	Average_Salary
IT	55300.000000
HR	55696.666667
Fin...	59250.000000
Sales	50000.000000
Mar...	44500.000000

Time	Action	Response	Duration / Fetch Time
16:03:29	SELECT * FROM EMPLOYEE WHERE Salary BETWEEN 40000 AND 70000 LIMIT 0, 1000	12 row(s) returned	0.010 sec / 0.000048...
16:04:48	SELECT * FROM EMPLOYEE WHERE Department IN ('IT', 'HR', 'Finance') LIMIT 0, 1000	11 row(s) returned	0.0060 sec / 0.00017...
16:06:47	SELECT * FROM EMPLOYEE WHERE Emp_Name LIKE 'A%' LIMIT 0, 1000	6 row(s) returned	0.0055 sec / 0.00019...
16:08:14	SELECT DISTINCT Department FROM EMPLOYEE LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.00041 s...
16:09:52	SELECT * FROM EMPLOYEE ORDER BY Salary DESC LIMIT 0, 1000	15 row(s) returned	0.0059 sec / 0.00015...
16:10:47	SELECT Department, COUNT(*) AS Employee_Count FROM EMPLOYEE GROUP BY Department LIMIT 0, 1000	5 row(s) returned	0.022 sec / 0.00069...
16:11:57	SELECT Department, AVG(Salary) AS Average_Salary FROM EMPLOYEE GROUP BY Department LIMIT 0, 1000	5 row(s) returned	0.0060 sec / 0.00018...

10. Apply the **HAVING** clause to display only those departments whose average salary is greater than ₹50,000.

Code

-- Display only departments whose average salary is greater than ₹50,000.

SELECT Department,

ROUND(AVG(Salary), 2) AS Average_Salary

FROM EMPLOYEE

GROUP BY Department

HAVING AVG(Salary) > 50000;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Find the number of employees in each department. Use COUNT() with GROUP BY.
SELECT Department, COUNT(*) AS Employee_Count
FROM EMPLOYEE
GROUP BY Department;

-- Calculate the average salary of employees in each department.
SELECT Department,
AVG(Salary) AS Average_Salary
FROM EMPLOYEE
GROUP BY Department;

-- You can round the result to two decimal places:
SELECT Department,
ROUND(AVG(Salary), 2) AS Average_Salary
FROM EMPLOYEE
GROUP BY Department;

-- Display only departments whose average salary is greater than ₹50,000.
SELECT Department,
ROUND(AVG(Salary), 2) AS Average_Salary
FROM EMPLOYEE
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

The Results tab shows the output of the last query:

Dep...	Average_Salary
IT	65000.00
HR	55666.67
Fin...	60250.00
Sales	50000.00
Mar...	54000.00

The Action Output tab shows the execution log:

Time	Action	Response	Duration / Fetch Time
16:08:46	SELECT * FROM EMPLOYEE WHERE Department IN ('IT', 'HR', 'Finance') LIMIT 0, 1000	11 row(s) returned	0.0060 sec / 0.00017...
16:08:47	SELECT * FROM EMPLOYEE WHERE Emp_Name LIKE 'A%' LIMIT 0, 1000	6 row(s) returned	0.0055 sec / 0.00019...
16:08:14	SELECT DISTINCT Department FROM EMPLOYEE LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.00041 s...
16:09:52	SELECT * FROM EMPLOYEE ORDER BY Salary DESC LIMIT 0, 1000	15 row(s) returned	0.0059 sec / 0.00015...
16:10:47	SELECT Department, COUNT(*) AS Employee_Count FROM EMPLOYEE GROUP BY Department LIMIT 0, 1000	5 row(s) returned	0.022 sec / 0.00069...
16:11:57	SELECT Department, AVG(Salary) AS Average_Salary FROM EMPLOYEE GROUP BY Department LIMIT 0, 1000	5 row(s) returned	0.0000 sec / 0.00018...
16:13:47	SELECT Department, ROUND(AVG(Salary), 2) AS Average_Salary FROM EMPLOYEE GROUP BY Department LIMIT 0, 1000	5 row(s) returned	0.011 sec / 0.00026 s...

11. Write a query using **WHERE, GROUP BY, HAVING, and ORDER BY** together to identify departments that:

- Have employees earning more than ₹40,000,
- Have at least 2 such employees, and
- Display the results according to the number of employees in descending order.

1. Prepare a brief comparison explaining the difference between **WHERE and HAVING clauses**, supported by SQL queries executed on the EMPLOYEE table.

Code

/*

Identify departments that:

Have employees earning more than ₹40,000

Have at least 2 such employees

Display results according to number of employees in descending order

*/

SELECT Department,

 COUNT(*) AS Employee_Count

FROM EMPLOYEE

WHERE Salary > 40000

GROUP BY Department

HAVING COUNT(*) >= 2

ORDER BY Employee_Count DESC;

MySQL Workbench interface showing a query execution session.

Query Editor:

```
107 -- Display only departments whose average salary is greater than $50,000.
108
109 SELECT Department,
110        ROUND(AVG(Salary), 2) AS Average_Salary
111 FROM EMPLOYEE
112 GROUP BY Department
113 HAVING AVG(Salary) > 50000;
114
115 /*
116 Identify departments that:
117 Have employees earning more than $40,000
118 Have at least 2 such employees
119 Display results according to number of employees in descending order
120 */
121
122 SELECT Department,
123        COUNT(*) AS Employee_Count
124 FROM EMPLOYEE
125 WHERE Salary > 40000
126 GROUP BY Department
127 HAVING COUNT(*) >= 2
128 ORDER BY Employee_Count DESC;
129
```

Result Grid:

Dep...	Employee_Count
IT	4
Fin...	4
Mar...	3
Mar...	2

Action Output:

Time	Action	Response	Duration / Fetch Time
16:08:47	SELECT * FROM EMPLOYEE WHERE Emp_Name LIKE 'A%' LIMIT 0, 1000	6 row(s) returned	0.0055 sec / 0.00018...
16:08:14	SELECT DISTINCT Department FROM EMPLOYEE LIMIT 0, 1000	5 row(s) returned	0.019 sec / 0.00041 s...
16:09:52	SELECT * FROM EMPLOYEE ORDER BY Salary DESC LIMIT 0, 1000	15 row(s) returned	0.0059 sec / 0.00015...
16:10:47	SELECT Department, COUNT(*) AS Employee_Count FROM EMPLOYEE GROUP BY Department LIMIT 0, 1000	5 row(s) returned	0.022 sec / 0.00069...
16:11:57	SELECT Department, AVG(Salary) AS Average_Salary FROM EMPLOYEE GROUP BY Department LIMIT 0, 1000	5 row(s) returned	0.0060 sec / 0.00018...
16:13:47	SELECT Department, ROUND(AVG(Salary), 2) AS Average_Salary FROM EMPLOYEE GROUP BY Department LIMIT 0, 1000	5 row(s) returned	0.011 sec / 0.00026 s...
16:16:58	SELECT Department, COUNT(*) AS Employee_Count FROM EMPLOYEE WHERE Salary > 40000 GROUP BY Department HAVING COUNT(*) >= 2 ORDER...	4 row(s) returned	0.0089 sec / 0.00017...

Query Completed

```
CREATE DATABASE employee_db;  
USE employee_db;
```

```
CREATE TABLE EMPLOYEE (  
    Emp_ID INT PRIMARY KEY,  
    Emp_Name VARCHAR(50) NOT NULL,  
    Department VARCHAR(30) NOT NULL,  
    Designation VARCHAR(50),  
    Salary DECIMAL(10,2) CHECK (Salary > 0),  
    City VARCHAR(30),  
    Joining_Date DATE  
);
```

```
INSERT INTO EMPLOYEE
```

```
(Emp_ID, Emp_Name, Department, Designation, Salary, City, Joining_Date)
```

```
VALUES
```

```
(101, 'Amit Sharma', 'IT', 'Software Engineer', 65000, 'Mumbai', '2022-01-15'),  
(102, 'Priya Singh', 'HR', 'HR Executive', 45000, 'Delhi', '2021-06-20'),  
(103, 'Rahul Verma', 'Finance', 'Financial Analyst', 58000, 'Bangalore', '2020-03-10'),  
(104, 'Ananya Gupta', 'IT', 'Data Analyst', 72000, 'Pune', '2023-02-18'),  
(105, 'Vikas Kumar', 'Sales', 'Sales Executive', 38000, 'Ranchi', '2022-08-12'),  
(106, 'Sneha Patel', 'Finance', 'Accountant', 48000, 'Ahmedabad', '2021-11-05'),  
(107, 'Arjun Mehta', 'IT', 'Backend Developer', 68000, 'Mumbai', '2022-04-25'),  
(108, 'Neha Roy', 'HR', 'Recruiter', 52000, 'Kolkata', '2020-09-15'),  
(109, 'Rohan Das', 'Marketing', 'Marketing Executive', 42000, 'Chennai', '2023-01-10'),  
(110, 'Ayesha Khan', 'Finance', 'Finance Manager', 75000, 'Hyderabad', '2019-07-22'),  
(111, 'Karan Joshi', 'IT', 'Frontend Developer', 55000, 'Delhi', '2021-02-14'),  
(112, 'Pooja Shah', 'Sales', 'Sales Manager', 62000, 'Mumbai', '2020-05-30'),  
(113, 'Aditya Rao', 'Marketing', 'Marketing Manager', 67000, 'Pune', '2019-12-01'),  
(114, 'Meera Nair', 'HR', 'HR Manager', 70000, 'Bangalore', '2018-10-19'),  
(115, 'Akash Jain', 'Finance', 'Senior Accountant', 56000, 'Jaipur', '2022-07-08');
```

-- 2. WHERE

SELECT *

FROM EMPLOYEE

WHERE Salary > 50000;

-- 3. BETWEEN

SELECT *

FROM EMPLOYEE

WHERE Salary BETWEEN 40000 AND 70000;

-- 4. IN

SELECT *

FROM EMPLOYEE

WHERE Department IN ('IT', 'HR', 'Finance');

-- 5. LIKE

SELECT *

FROM EMPLOYEE

WHERE Emp_Name LIKE 'A%';

-- 6. DISTINCT

SELECT DISTINCT Department

FROM EMPLOYEE;

-- 7. ORDER BY

SELECT *

FROM EMPLOYEE

ORDER BY Salary DESC;

-- 8. GROUP BY + COUNT

```
SELECT Department,  
       COUNT(*) AS Employee_Count  
FROM EMPLOYEE  
GROUP BY Department;
```

-- 9. GROUP BY + AVG

```
SELECT Department,  
       ROUND(AVG(Salary), 2) AS Average_Salary  
FROM EMPLOYEE  
GROUP BY Department;
```

-- 10. GROUP BY + HAVING

```
SELECT Department,  
       ROUND(AVG(Salary), 2) AS Average_Salary  
FROM EMPLOYEE  
GROUP BY Department  
HAVING AVG(Salary) > 50000;
```

-- 11. WHERE + GROUP BY + HAVING + ORDER BY

```
SELECT Department,  
       COUNT(*) AS Employee_Count  
FROM EMPLOYEE  
WHERE Salary > 40000  
GROUP BY Department  
HAVING COUNT(*) >= 2  
ORDER BY Employee_Count DESC;
```